Lecture Notes: Transformer Architectures

Course: 2110573 Pattern Recognition

2026-02-19
version 0.1

1 Introduction to the Transformer

The Transformer is a novel network architecture introduced in the 2017 paper "Attention Is All You Need" by Vaswani et al. from Google. It was a significant departure from the recurrent neural network (RNN) based models that were state-of-the-art for sequence transduction tasks like machine translation at the time.

The core idea of the Transformer is to rely exclusively on **attention mechanisms**, completely discarding recurrence and convolutions. This design has two primary advantages:

- **Increased Parallelization:** By removing the sequential nature of RNNs, computations over a sequence can be performed in parallel, leading to significantly faster training times.
- **Improved Long-Range Dependency Handling:** The attention mechanism allows the model to directly connect any two positions in a sequence, regardless of their distance. This makes it more effective at capturing long-range dependencies compared to RNNs, where information has to pass through many intermediate steps.

2 High-Level Architecture of Vanilla Transformer

The Transformer follows a standard **encoder-decoder** structure, which is common in sequence-to-sequence tasks.

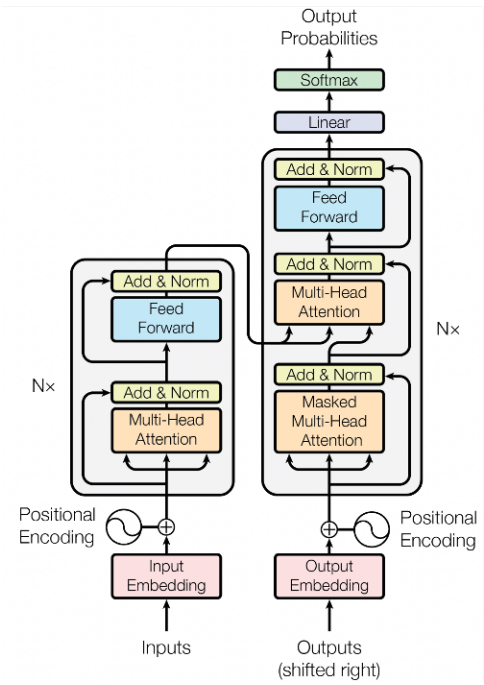

Figure 1: The Transformer Architecture, showing the stack of N=6 encoders and decoders. Source: "Attention is All You Need".

2.1 The Encoder-Decoder Stack

The architecture consists of a stack of identical encoders and a stack of identical decoders. In the original paper, the number of layers (N) in each stack is 6.

- **The Encoder:** Its role is to map an input sequence of symbol representations $(x_1, \ldots, x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, \ldots, z_n)$.
- **The Decoder:** Given $\mathbf{z}$, the decoder generates an output sequence $(y_1, \ldots, y_m)$ one element at a time. It is auto-regressive, meaning it consumes the previously generated symbols as additional input when generating the next one.

2.2 Layer Composition

Each encoder and decoder layer is composed of a few key sub-layers.

- **Encoder Layer:** Each encoder layer has two sub-layers:
  1. A multi-head self-attention mechanism.
  2. A simple, position-wise fully connected feed-forward network.
- **Decoder Layer:** Each decoder layer has three sub-layers:
  1. A **masked** multi-head self-attention mechanism to prevent positions from attending to subsequent positions.
  2. A multi-head **encoder-decoder attention** mechanism, which allows every position in the decoder to attend over all positions in the input sequence.
  3. A position-wise fully connected feed-forward network.

A residual connection followed by layer normalization is applied around each of these sub-layers.

3 The Attention Mechanism

Attention can be described as a function that maps a query and a set of key-value pairs to an output. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

3.1 Scaled Dot-Product Attention

The specific attention mechanism used in the Transformer is called Scaled Dot-Product Attention. For a query Q, keys K, and values V, the attention is calculated as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1}$$

Where d_k is the dimension of the keys. The scaling factor $\sqrt{d_k}$ is crucial for stabilizing gradients, as the dot products can grow large for high-dimensional vectors, pushing the softmax function into regions with extremely small gradients.

3.2 Attention Variations in the Transformer

The Transformer uses three different types of attention:

1. **Self-Attention (Encoder):** Here, the queries, keys, and values (Q, K, V) all come from the same place: the output of the previous layer in the encoder. This allows each position in the encoder to attend to all other positions in the previous layer's output, effectively calculating a representation of the sequence by relating its own words to each other.

2. **Masked Self-Attention (Decoder):** In the decoder's self-attention layer, queries, keys, and values also come from the same source (the previous decoder layer). However, to preserve the auto-regressive property (i.e., prediction of the current token can only depend on previous tokens), future positions are masked out. This is implemented by setting the softmax input for illegal connections to $-\infty$.

3. **Encoder-Decoder Attention (Decoder):** This is also known as "cross-attention". Here, the queries Q come from the previous decoder layer, while the keys K and values V come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence, which is how the decoder focuses on relevant parts of the source sentence.

4 Multi-Head Attention

Instead of performing a single attention function, the authors found it beneficial to linearly project the queries, keys, and values multiple times with different, learned linear projections. This is called Multi-Head Attention. It allows the model to jointly attend to information from different representation subspaces at different positions.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{2}$$

where each head is calculated as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \tag{3}$$

The process is as follows:

1. The input Q, K, and V matrices are multiplied by learned weight matrices (W_i^Q, W_i^K, W_i^V) for each of the h attention heads.

2. Scaled dot-product attention is applied in parallel to each of these projected versions of Q, K, and V, yielding h output matrices $(Z_0, Z_1, \ldots, Z_{h-1})$.

3. These output matrices are concatenated.

4. The concatenated matrix is multiplied by another learned weight matrix W^O to produce the final output of the multi-head attention layer.

In the base Transformer model, $h = 8$.

5 Positional Encodings

Since the model contains no recurrence or convolution, it has no inherent sense of the order or position of the tokens in the input sequence. To address this, "positional encodings" are added to the input embeddings at the bottoms of the encoder and decoder stacks. These encodings are vectors that give the model information about the position of a token in the sequence.

The authors used sine and cosine functions of different frequencies:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}}) \tag{4}$$

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}}) \tag{5}$$

where pos is the position of the token in the sequence, i is the dimension within the embedding vector, and d_{model} is the model's embedding dimension. This choice allows the model to easily learn to attend by relative positions, since for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}.

There are other types of positional encoding, such as learned absolute encodings (used in GPT) and relative positional encodings (used in GPT-NeoX).

6 Add & Norm: Residuals and Layer Normalization

Each sub-layer (self-attention, feed-forward network) in the encoder and decoder has a residual connection around it, followed by a layer normalization step. The output of each sub-layer is thus $\text{LayerNorm}(x + \text{Sublayer}(x))$.

Layer Normalization differs from Batch Normalization. Layer Normalization computes the mean and variance used for normalization from all of the summed inputs to the neurons in a single layer for a single training example. In contrast, Batch Normalization computes these statistics across the mini-batch for each feature independently. In the context of NLP, this means Layer Normalization normalizes across the features of each word vector independently for each sample in the batch.

7 Final Output Generation

The decoder stack outputs a vector of floats. To turn this vector into an output token, a final linear layer followed by a softmax function is used.

- **Linear Layer:** A fully connected neural network that projects the vector produced by the decoder stack into a much larger vector, known as a logits vector. The size of this vector is equal to the size of the vocabulary.
- **Softmax Layer:** The softmax function turns the logit scores into probabilities. The cell with the highest probability is chosen, and the word associated with it is produced as the output token for that time step.

8 The Transformer Family: Architectural Variants

Since its inception, the Transformer architecture has been adapted into three main variants, each suited for different types of tasks.

8.1 Encoder-Only Models (Auto-Encoders)

These models use only the encoder stack of the Transformer. Because they lack a decoder, they are not naturally suited for generation. However, their self-attention mechanism is fully bidirectional, meaning every token can attend to every other token in the sequence (both past and future).

- **Examples:** BERT, ALBERT, RoBERTa.
- **Use Cases:** Excellent for understanding content. Used in tasks like text classification, named entity recognition (NER), and question answering.
- **Pre-training Objective:** Typically trained on a **Masked Language Model (MLM)** objective, where some percentage of input tokens are masked, and the model must predict the original tokens.

8.2 Decoder-Only Models (Autoregressive)

These models use only the decoder stack of the Transformer. The self-attention mechanism is "causal" or "masked," meaning a token can only attend to itself and previous tokens in the sequence. This makes them inherently suitable for generating sequences from left to right.

- **Examples:** GPT series, Llama, BLOOM.
- **Use Cases:** Primarily used for text generation, language modeling, and building chatbots.
- **Pre-training Objective:** Trained to predict the **next token** in a sequence given all previous tokens.

8.3 Encoder-Decoder Models (Sequence-to-Sequence)

These models use the full architecture of the original Transformer. They are ideal for tasks that require transforming an input sequence into a new output sequence.

- **Examples:** T5, BART, MASS, the original Transformer.
- **Use Cases:** Machine translation, text summarization, question answering.
- **Pre-training Objective:** Varies, but often involves corrupting the input sequence in some way (e.g., masking spans of text as in T5 and BART) and training the decoder to reconstruct the original, uncorrupted sequence.

9 Challenges and Optimizations

The original Transformer architecture, while powerful, has several limitations, primarily related to its computational cost and context length. The quadratic complexity of the attention mechanism ($O(L^2)$ for a sequence of length L) is a major bottleneck.

9.1 Parameter Efficiency: ALBERT

ALBERT ("A Lite BERT") tackles the large number of parameters in models like BERT. Its two main innovations are:

- **Factorized Embedding Parameterization:** It decouples the size of the vocabulary embedding (V) from the hidden layer size (H). Instead of a large embedding matrix of size $V \times H$, it uses two smaller matrices of size $V \times E$ and $E \times H$, where $E \ll H$. This reduces the parameter count from $O(V \times H)$ to $O(V \times E + E \times H)$.
- **Cross-Layer Parameter Sharing:** It shares all parameters (feed-forward networks and attention) across all layers, which dramatically reduces the total number of unique parameters and stabilizes training.

9.2 Extending Context: Rotary Position Embedding (RoPE)

Standard absolute or learned positional embeddings do not generalize well to sequence lengths longer than those seen during training. **Rotary Position Embedding (RoPE)** provides a novel way to encode position that has become central to modern LLMs like Llama and PaLM.

Instead of adding positional information to the embeddings, RoPE *modifies* the query and key vectors by rotating them based on their absolute position. The key insight is to perform this rotation in a way that the subsequent dot product for attention only depends on the *relative* distance between the tokens.

Mechanism: RoPE treats the d-dimensional embedding vector as a sequence of $d/2$ two-dimensional vectors. Each of these 2D vectors is rotated by an angle that depends on its position in the sequence. For a query vector $\mathbf{q}_m$ at position m, the rotation is applied as follows:

$$f(\mathbf{q}, m) = \mathbf{R}_m \mathbf{q} \tag{6}$$

where $\mathbf{R}_m$ is a rotation matrix that depends on the position m. This matrix operates on pairs of features, mixing them based on a rotation angle $\theta_i \cdot m$:

$$\begin{pmatrix} q'_{m,2i} \\ q'_{m,2i+1} \end{pmatrix} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \begin{pmatrix} q_{m,2i} \\ q_{m,2i+1} \end{pmatrix} \tag{7}$$

The same rotation is applied to the key vector $\mathbf{k}_n$ at position n.

The Key Property: The magic of RoPE lies in the dot product. Due to the properties of rotation matrices, the dot product between the rotated query at position m and the rotated key at position n is equivalent to a rotation of their original dot product by an angle dependent on their relative position, $m - n$.

$$\langle f(\mathbf{q}, m), f(\mathbf{k}, n)\rangle = \langle \mathbf{R}_m \mathbf{q}, \mathbf{R}_n \mathbf{k}\rangle = \langle \mathbf{q}, \mathbf{R}_{m-n} \mathbf{k}\rangle \tag{8}$$

This means the attention score intrinsically depends on the relative distance, not the absolute positions, allowing the model to generalize to much longer sequences than it was trained on. It also has the desirable property that the attention strength naturally decays as the relative distance between tokens increases.

9.3 Computational Efficiency

9.3.1 Sparse Attention

Instead of computing the full, dense attention matrix, sparse attention methods compute attention only for a subset of token pairs. Models like **BigBird** combine several sparse attention patterns:

- **Window Attention:** Each token attends to a local window of neighboring tokens.
- **Global Attention:** A few special tokens (like '[CLS]') attend to all other tokens.
- **Random Attention:** Each token attends to a small, random set of other tokens.

This combination approximates the full attention matrix while reducing the complexity to be linear with sequence length.

9.3.2 FlashAttention

This is not an approximation but a hardware-aware optimization for computing exact attention. The standard implementation of attention involves writing the large $L \times L$ attention matrix to the GPU's high-bandwidth memory (HBM), which is slow. **FlashAttention** restructures the computation to avoid this. It loads blocks of the Q, K, and V matrices into the much faster on-chip SRAM, computes attention for that block, and writes only the final result back to HBM. It uses techniques like tiling and recomputation to make this possible, leading to significant speedups (2-4x) and memory savings. **FlashAttention-2** further optimizes this by improving parallelism on the GPU.

9.3.3 Linear Attention

These methods aim to reduce the computational complexity of the attention mechanism from quadratic ($O(L^2)$) to linear ($O(L)$) with respect to sequence length L. This is achieved by avoiding the explicit computation of the massive $L \times L$ attention matrix.

The Associative Trick: The standard attention computation, Attention $= \text{softmax}(QK^T)V$, is bottlenecked by the matrix multiplication QK^T. Linear attention methods work by replacing the softmax function with a kernelized similarity function, $\text{sim}(Q, K)$, that can be expressed as the dot product of feature maps ϕ:

$$\text{sim}(Q, K) = \phi(Q)\phi(K)^T \tag{9}$$

A common choice for the feature map is the Exponential Linear Unit (ELU) plus one, $\phi(x) = \text{elu}(x) + 1$, to ensure positivity.

This decomposition allows for a crucial change in the order of operations due to the associative property of matrix multiplication. Instead of computing the full attention matrix first:

$$\text{Output} = (\phi(Q)\phi(K)^T)V \tag{10}$$

We can re-associate the terms like this:

$$\text{Output} = \phi(Q)(\phi(K)^T V) \tag{11}$$

Analyzing the complexity of the second approach reveals the linear scaling. The term $(\phi(K)^T V)$ involves multiplying a $(d \times L)$ matrix with an $(L \times d_v)$ matrix, resulting in a small $(d \times d_v)$ matrix. This computation is $O(L \cdot d \cdot d_v)$. The final multiplication of $\phi(Q)$ (an $(L \times d)$ matrix) with this result is also linear in L. The overall complexity is therefore $O(L)$, a dramatic improvement.

Recurrent Formulation and Inference: A significant advantage of this formulation is its equivalence to a Recurrent Neural Network (RNN). The global context vector $S_i = \sum_{j=1}^{i} \phi(\mathbf{k}_j)^T \mathbf{v}_j$ can be computed iteratively:

$$S_i = S_{i-1} + \phi(\mathbf{k}_i)^T \mathbf{v}_i \tag{12}$$

The output at step i is then simply $\mathbf{o}_i = \phi(\mathbf{q}_i) S_i$. This means that during autoregressive inference (generating one token at a time), the state S_{i-1} can be cached and updated in constant time ($O(1)$) to compute the next output. This is a massive speedup compared to standard Transformers, where the entire Key-Value cache for all previous tokens must be re-processed at each step, an $O(L)$ operation.

Trade-offs: While extremely efficient, linear attention is an approximation of the powerful softmax attention. The exponential nature of the softmax function allows the model to sharply focus on a few key tokens, effectively ignoring others. Most linear attention kernels are "softer" and do not replicate this property perfectly, which can sometimes lead to a degradation in model performance compared to their exact attention counterparts. Modern architectures today will interleave linear attention layers and softmax attention layers together to reduce the downside.

9.4 Normalization Improvements

- **Pre-LN vs. Post-LN:** The original Transformer used Post-Layer Normalization (applying LN after the residual connection), which can lead to unstable training. Most modern Transformers use **Pre-LN** (applying LN *before* the sub-layer), which stabilizes training and often removes the need for learning rate warm-up.
- **RMS Norm:** Root Mean Square Layer Normalization is a simplified version of LayerNorm. It normalizes the vector by its root-mean-square, but omits the re-scaling step based on learnable parameters. It is more computationally efficient and has been shown to be just as effective in large language models.

10 Transformers in Vision (ViT)

The Transformer's success has been extended to computer vision.

- **Vision Transformer (ViT):** The core idea is to treat an image as a sequence of fixed-size patches (e.g., 16x16 pixels). These patches are flattened and linearly projected to create "token" embeddings. Paired with positional embeddings, this sequence of tokens is fed into a standard Transformer encoder for a classification task. ViTs have shown that a reliance on convolution is not necessary for state-of-the-art vision results, though they typically require very large pre-training datasets.
- **Swin Transformer:** Swin improves upon ViT by reintroducing some useful inductive biases from CNNs. It computes attention locally within non-overlapping windows and builds a hierarchical representation (feature maps get smaller in deeper layers). To allow for cross-window communication, the windows are shifted in subsequent layers. This design achieves linear computational complexity with respect to image size and performs strongly on a wider range of vision tasks.

11 Beyond Transformers: State-Space Models (SSM)

While Transformers are dominant, research into architectures that scale linearly with sequence length while retaining strong performance is active. State-Space Models are a leading alternative.

- **Concept:** SSMs are inspired by control theory and are fundamentally a type of RNN. A continuous signal $u(t)$ is mapped to a representation $y(t)$ via a latent state $x(t)$.
- **S4 (Structured State Space for Sequence Modeling):** The problem with RNNs is their slow, sequential nature. S4 solves this by showing that for discrete, time-invariant systems, the entire recurrent process can be unrolled into a single, large **convolutional kernel**. This convolution can be computed very quickly in parallel using Fast Fourier Transforms (FFT). A special initialization of the state matrix (HiPPO) allows it to effectively model long-range dependencies.

- **Mamba:** Mamba improves upon S4. While S4's convolutional filter is input-agnostic, Mamba introduces a **Selective Scan (S6)** mechanism. Its state-space parameters are derived from the input itself, making them time-varying and input-dependent. This allows Mamba to selectively focus on or ignore parts of the input, mimicking the adaptive behavior of attention. Mamba achieves linear scaling in sequence length during training and constant time ($O(1)$) per step during inference, while being competitive with Transformer models.
- **Hybrid Models (Jamba):** The latest trend is to combine the strengths of different architectures. **Jamba** is a hybrid model that interleaves blocks of Mamba layers with traditional Transformer attention layers, aiming to get the best of both worlds: the efficiency of Mamba and the proven performance of attention.